

Глубинное обучение в обработке звука

Neural Codecs

План лекции

- Сжатие аудио

- Алгоритмы квантования

- Нейросетевые кодеки

Сжатие аудио

Сжатие аудио

- «Сырой» звук весит достаточно много
- В *wav* формате 1 минута аудио занимает ≈ 10 Мб

Существует 2 способа получить нужный файл:

- **Скачивание** – ждём, пока весь файл скачается и потом открываем
- **Потоковая передача (*Streaming*)** – файл воспроизводится сразу, пока остальные данные догружаются

Скачивание

Скачивание нужно:

- Для установки игр и программ
- Для отправки документов
- Для оффлайн режима

- При скачивании важен размер файла
- Если медленный интернет, то скачивание будет идти дольше
- Медленный интернет не повлияет на качество файла

Streaming

Streaming нужен:

- Для видеосервисов: *YouTube, Netflix, TikTok*
- Для музыкальных сервисов: *Spotify, Apple Music*
- Для видеозвонков и конференций: *Zoom, Skype, Google Meet*
- Для прямых эфиров

- При стриминге важна скорость интернета
- Эта скорость называется ***bitrate***

Bitrate

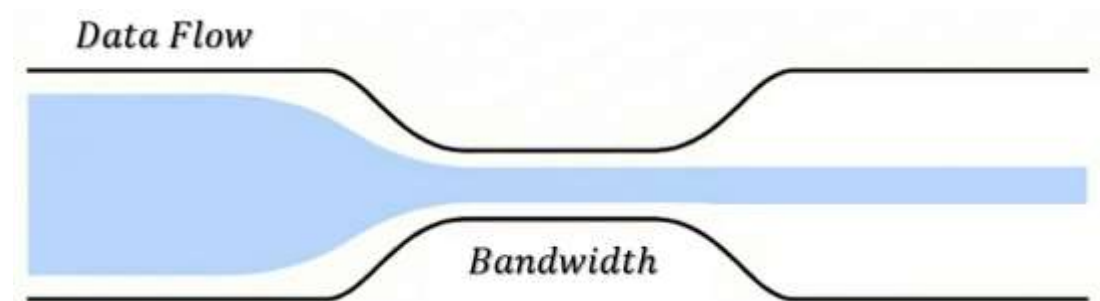
Bitrate – количество информации, передаваемое за единицу времени

$$\text{Bitrate} = \text{Sample rate} \times \text{Depth} \times \text{Channels}$$

Чтобы слушать *wav* файл без задержек необходимо:

$$\text{Bitrate} = 44100 \times 16 \times 2 = 1\,411\,200 \approx 1\,411 \text{ Кбит/с}$$

- У всех каналов связи есть **пропускная способность**
- В эпоху 2G/3G пропускная способность могла падать до 50 – 100 Кбит/с



Нужно уменьшить *bitrate* потока в 10 – 20 раз и при этом сохранить качество

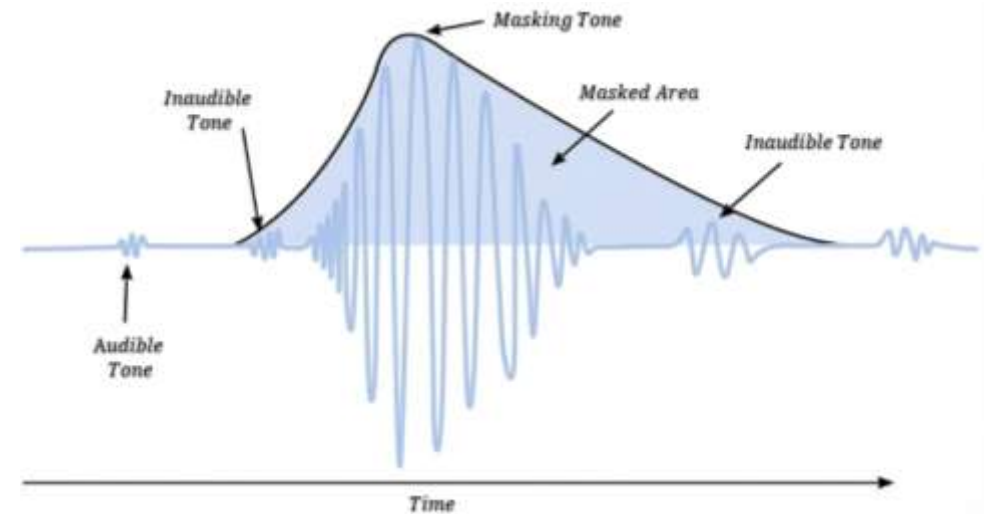
MP3

Цель:

Хотим сжать аудио в 10 раз, чтобы передавать через медленный интернет 90-ых

Чистая математика бессильна – аудио слишком хаотично

- Звуковая волна режется на множество фреймов
- Каждый фрейм раскладывается в спектр
- Алгоритм анализирует спектр и ищет то, что человек не слышит



FLAC

Цель:

Хотим сделать сжатие без потерь, уменьшив размер файла в несколько раз

- Аудио разбивается на блоки и алгоритм пытается предсказать следующий отсчет на основе предыдущих
- В файл записывается не сам сигнал, а ошибка предсказания
- Ошибка занимает гораздо меньше места

Opus и EVS

Используются в стриминговых сервисах и телефонии

- Внутри кодека есть классификатор, который решает, что сейчас сжимать – речь или музыку
 - Для речи включается модель голосового тракта *LPC*, для музыки – *MDCT*
-
- *Opus* используется для звонков через интернет, стриминга
 - *EVS* используется для телефонии (*VoLTE*)

Предел

- *MP3* и *OPUS* достигли своего предела
- Ниже 10-20 Кбит/с алгоритму приходится удалять много важной информации

Это привело к появлению нейросетевых кодеков

Нейросетевые кодеки

Любой нейросетевой кодек состоит из 3 частей:

- Кодировщик
- Блок квантования
- Декодировщик



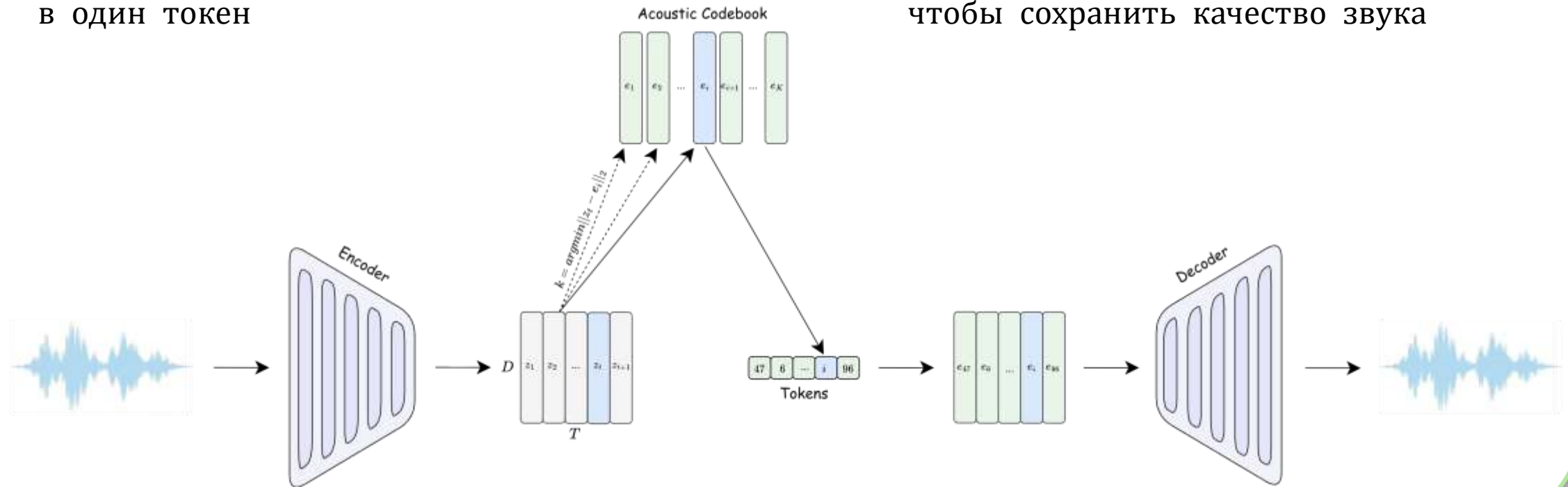
- **Кодировщик** преобразует входной сигнал в непрерывный скрытый вектор
- **Блок квантования** преобразует непрерывный вектор в набор дискретных индексов
- **Декодировщик** восстанавливает исходный сигнал из квантованных векторов

Алгоритмы квантования

Single Vector Quantization

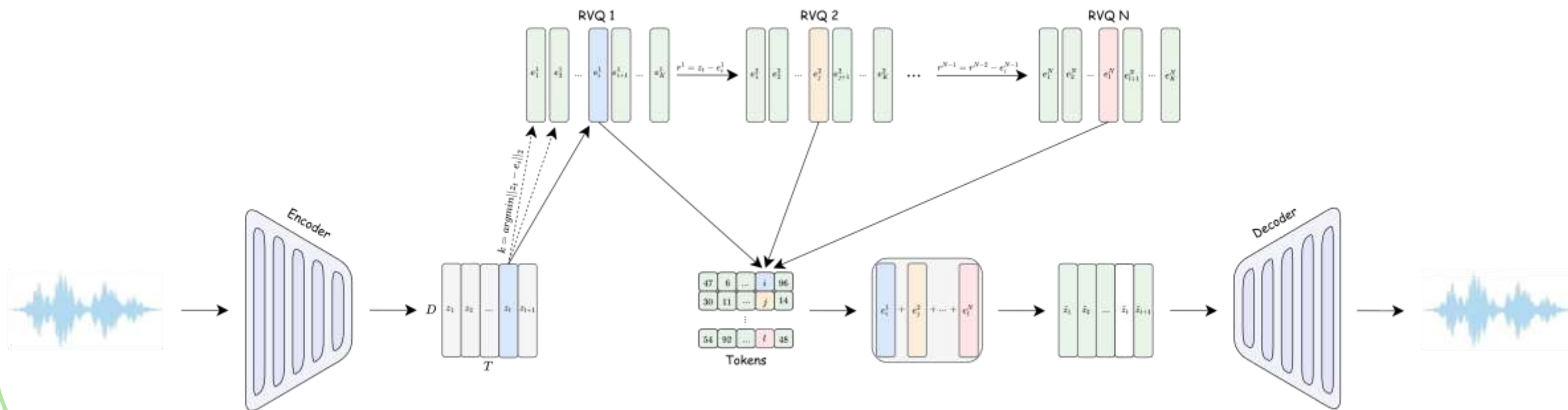
- Одна кодовая книга
- Вектор признаков отображается в один токен

- Простая архитектура
- Нужен огромный размер словаря, чтобы сохранить качество звука



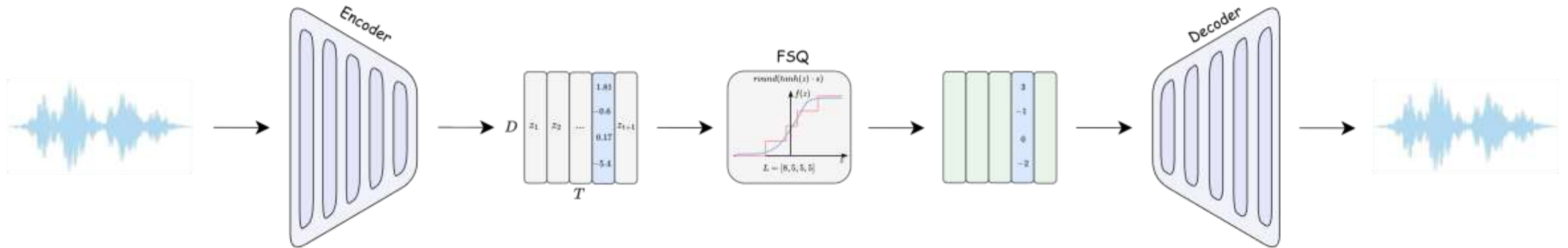
Residual Vector Quantization

- Несколько кодовых книг
- RVQ 1 кодирует сигнал, а каждый следующий кодирует ошибку (*residual*) предыдущего
- Возможность менять битрейт
- Высокое качество звука с небольшими словарями



Finite Scalar Quantization

- Не используются словари, значения векторов просто округляются
- Каждое измерение вектора округляется своим количеством уровней L
- Эффективный размер словаря – это произведение уровней L
- Мгновенная операция – нет поиска ближайших соседей



Нейросетевые кодеки

SoundStream, 2021

Проблемы прошлых моделей:

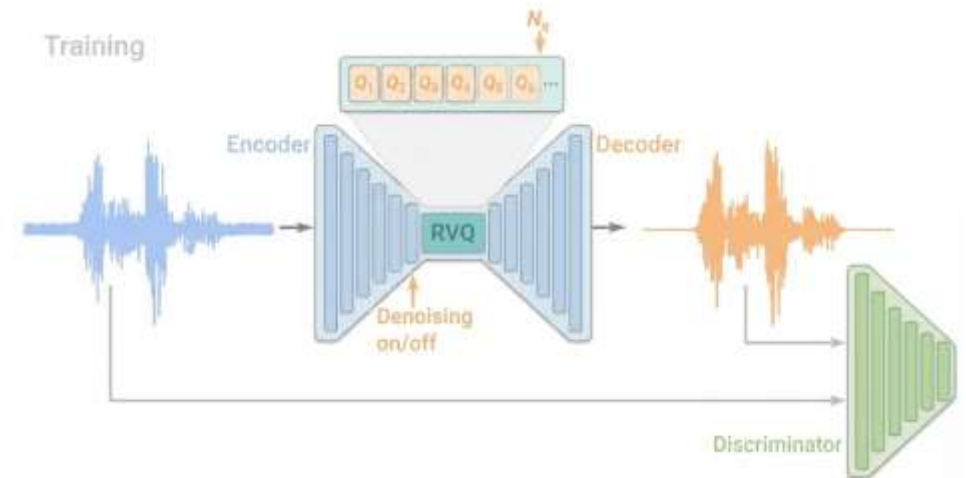
- *Opus* и *EVS* на низких битрейтах имеют заметные артефакты
- Большинство кодеков работают в узкой области (речь \ музыка)
- Для каждого битрейта нужно учить свою модель

Идея SoundStream:

- Будем совместно обучать *Encoder + Quantizer + Decoder*

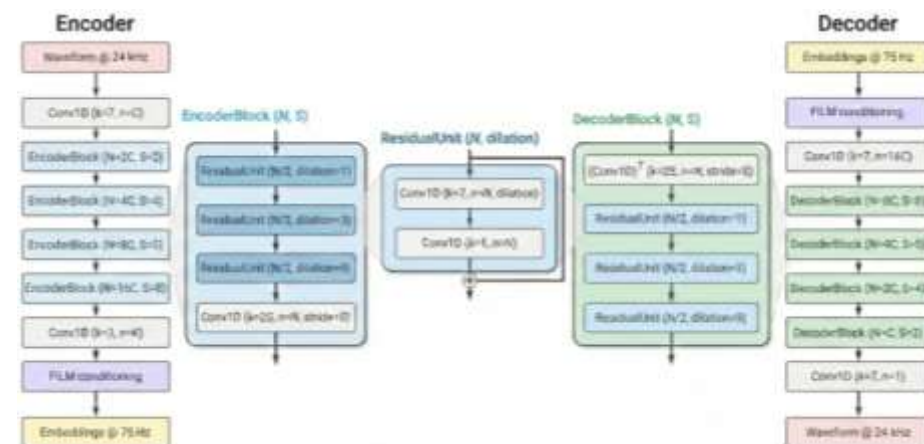
Архитектура SoundStream

- **Encoder** преобразует в последовательность эмбеддингов
- Вместо большой кодовой книги используется **RVQ**
- **Decoder** восстанавливает аудио из квантованных векторов
- **Discriminator** учится отличать реальное аудио от сгенерированного



SoundStream: Encoder и Decoder

- **Encoder** состоит из B_{enc} сверточных блоков + 1D свёртки на входе и выходе
- Для работы в **streaming** используются каузальные свёртки
- **Decoder** симметричен **Encoder**'у
- **Upsampling** происходит с помощью транспонированных свёрток
- Можно сделать **Encoder** или **Decoder** легче, изменив C_{enc} или C_{dec}



SoundStream: RVQ

- Для битрейта 6 кбит/с (80 бит на фрейм) потребовался бы *codebook* размером 2^{80} векторов
- Будем использовать *RVQ*
- Обновление весов кодовых книг происходит через скользящее среднее (*EMA*)
- Если вектор из книги долго не используется, он заменяется на случайный вектор из батча
- Во время обучения модель использует случайное число *RVQ* слоёв

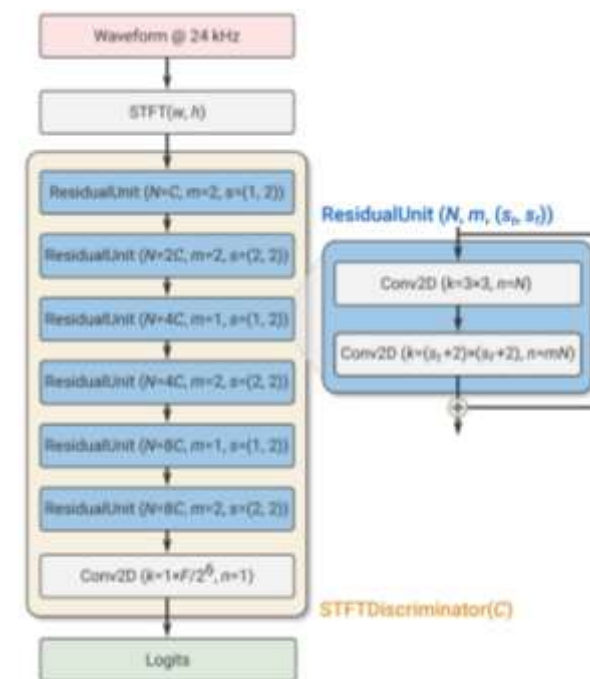
SoundStream: Discriminator

Waveform Discriminator:

- Работает с 3 сигналами: оригинал и его сжатые версии в 2 и 4 раза
- Для каждого сигнала используется свой дискриминатор

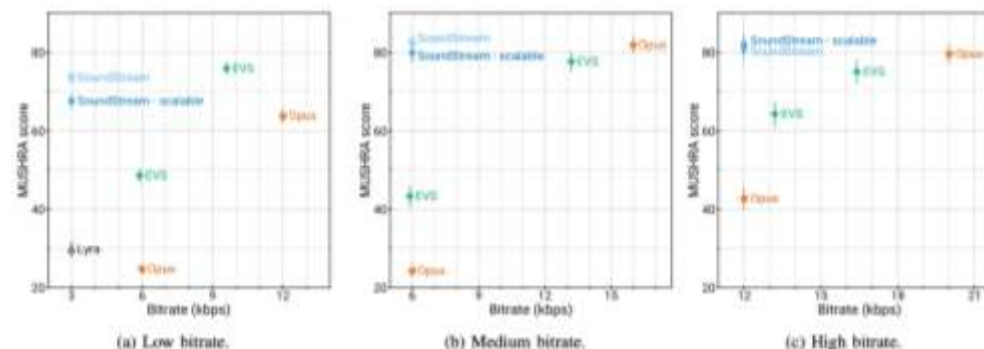
STFT Discriminator:

- Анализирует *STFT*
- Состоит из 2D свёрток и 6 *ResidualUnit* с разными *stride*
- Помогает генератору создавать чистый звук без металлических эффектов



Результаты *SoundStream*

- Модель училась на датасетах *LibriTTS*, *FreeSound*, *MagnaTagATune*
- На 3 кбит/с звучит лучше, чем *Opus* на 12 кбит/с и сопоставим с *EVS* на 9.6 кбит/с
- Одинаково хорошо справляется и с речью, и с музыкой
- Может работать с разными битрейтами
- Есть возможность удалять фоновые шумы



Encodec, 2022

Проблемы прошлых моделей:

- Используют ограниченные алгоритмы цифровой обработки сигналов (*Opus, EVS*)
- Имеют предел эффективности сжатия, особенно на низких битрейтах
- Нужно балансировать множество функций потерь и гиперпараметров

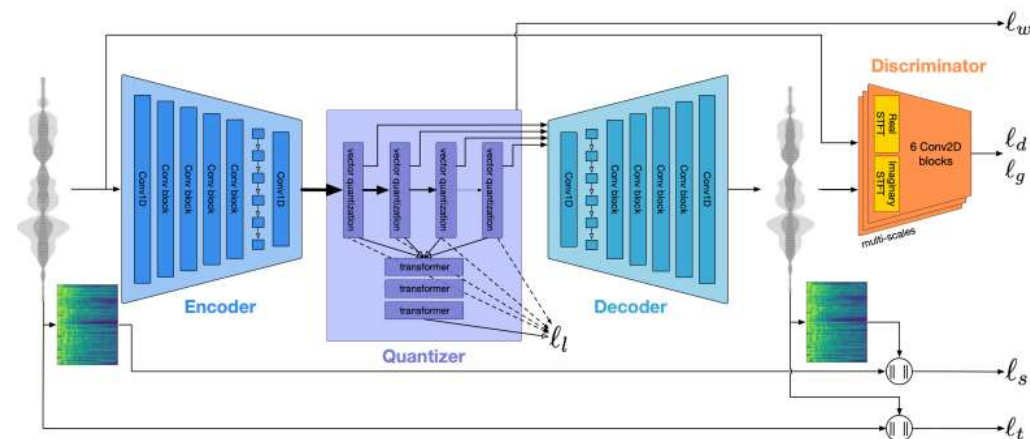
Идея Encodec:

Будем учить *end-to-end* модель с балансировщиком градиентов

Архитектура *Encodec*

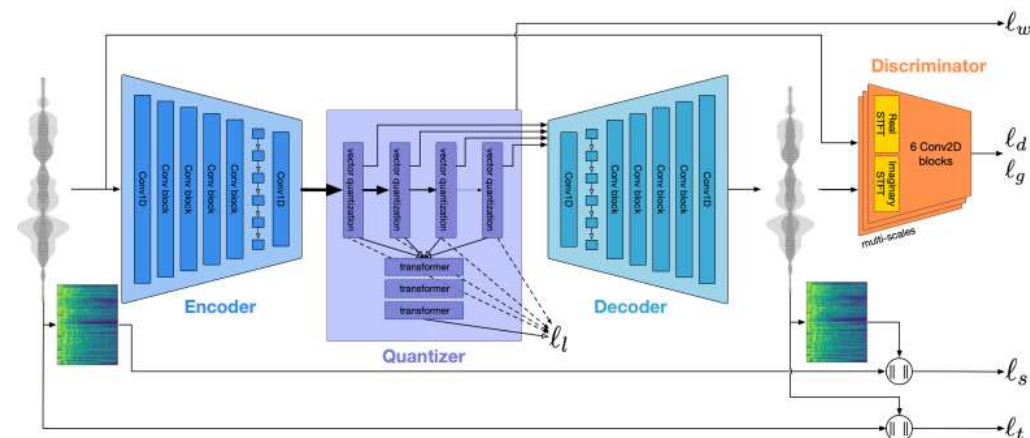
Модель учится *end-to-end* и состоит из 3 частей:

- ***Encoder*** преобразует аудио в скрытое представление $\mathbf{z}$
- ***Quantizer*** с помощью *RVQ* сжимает $\mathbf{z}$ в дискретный набор кодов $\mathbf{z}_q$
- ***Decoder*** восстанавливает аудио из кодов



Encodec: Encoder и Decoder

- **Encoder** состоит из $B = 4$ сверточных блоков + 1D свёртки на входе и выходе
- После 4 сверточных блоков используется *LSTM* для моделирования длинных зависимостей
- **Decoder** является копией *Encoder*'а, но использует транспонированные свёртки для *Upsampling*
- Модель поддерживает *streaming* и *non-streaming* режимы



Encoder: RVQ

- Используется до 32 словарей с 1024 векторами (10 бит)
- Битрейт регулируется количеством используемых словарей
- Модель во время обучения выбирает случайное и кратное 4 значение словарей N_q
- Это позволяет модели поддерживать битрейты 1.5, 3, 6, 12 и 24 кбит/с
- Для обновления словарей во время обучения используется *EMA*
- Неиспользуемые коды заменяются случайными векторами из текущего батча
- Используется *commitment loss*, чтобы притягивать выход кодировщика к векторам из словаря

Encoder: Entropy Encoding

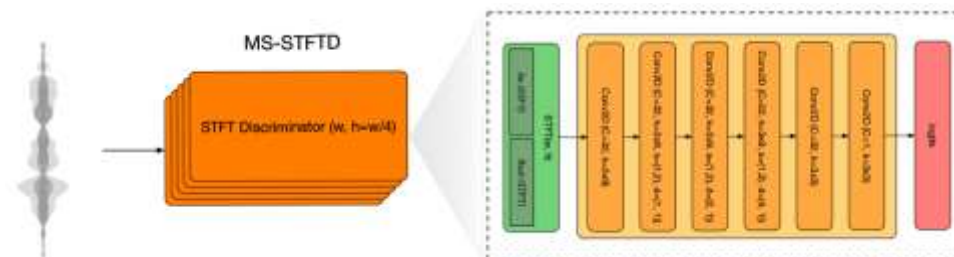
- Для сжатия еще на 40% используется *энтропийное кодирование (Entropy Encoding)*
- Квантованные коды имеют статистические зависимости и мы можем предсказывать следующий код, зная предыдущие
- Обучается лёгкий трансформер (5 слоёв, 8 голов) для предсказания вероятности кодов *RVQ*
- На основе предсказаний трансформера применяется арифметическое кодирование

Обучение *Encodec*

- Вместо сложной комбинации разных дискриминаторов используется единая архитектура ***Multi – Scale STFT Discriminator***
- Во время обучения используется 5 субдискриминаторов с разными длинами окон [2048, 1024, 512, 256, 128]
- Функция потерь состоит из 5 слагаемых:

$$L = L_{time} + L_{freq} + L_{Adv} + L_{Feat} + L_{VQ}$$

- Вместо подбора коэффициентов λ происходит нормировка градиентов и λ показывает долю участия в обучении



Результаты *Encodec*

- Модель училась на разнообразных датасетах речи и музыки
- Превосходит *Opus* на 12 кбит/с и *Lyra-v2* на 6 кбит/с
- Использование трансформера помогает снизить полосу пропускания до 40%
- Работает в *real-time* на одном ядре *CPU*
- Может работать на разных битрейтах и видах аудио

Model	Bandwidth	Entropy Coded	Clean Speech	Noisy Speech	Music Set-1	Music Set-2
Reference	-	-	95.5±1.6	93.9±1.8	93.2±2.5	97.1±1.3
Opus	6.0 kbps	-	30.1±2.8	19.1±5.9	20.6±5.8	17.9±5.3
Opus	12.0 kbps	-	76.5±2.3	61.9±2.1	77.8±3.2	65.4±2.7
EVS	9.6 kbps	-	84.4±2.5	80.0±2.4	89.9±2.3	87.7±2.3
Lyra-v2	3.0 kbps	-	53.1±1.9	52.0±4.7	69.3±3.3	42.3±3.5
Lyra-v2	6.0 kbps	-	66.2±2.9	59.9±3.3	75.7±2.6	48.6±2.1
ENCODEC	1.5 kbps	0.9 kbps	49.2±2.4	41.3±3.6	68.2±2.2	66.5±2.3
ENCODEC	3.0 kbps	1.9 kbps	67.0±1.5	62.5±2.3	89.6±3.1	87.8±2.9
ENCODEC	6.0 kbps	4.1 kbps	83.1±2.7	69.4±2.3	92.9±1.8	91.3±2.1
ENCODEC	12.0 kbps	8.9 kbps	90.6±2.6	80.1±2.5	91.8±2.5	92.9±1.2

Descript Audio Codec, 2023

Проблемы прошлых моделей:

- Кодеки на основе *GAN* часто страдают от артефактом и металлического звука
- В моделях с векторным квантованием значительная часть векторов не используется
- *Quantization Dropout* ухудшает качество звука на полной полосе пропускания

Идея Descript Audio Codec:

- Обучим *VQ-GAN* архитектуру с *improved RVQ* и *Dropout*

Архитектура DAC

- Архитектура имеет вид *VQ-GAN* с полностью сверточными *encoder* и *decoder + RVQ*
- Для моделирования периодических сигналов используется функция активации *snake*:

$$Snake(x) = x + \frac{1}{\alpha} \sin^2(\alpha x)$$

- Искать ближайший вектор в кодовой книге в 1024 размерном пространстве сложно из-за проклятия размерности
- Будем проецировать вектор в пространство меньшей размерности и в нём искать соседа
- Найдя индекс ближайшего соседа, используем соответствующий ему «большой» вектор
- Все векторы нормализуются и расстояние между ними превращается в косинусную близость и все векторы имеют равные шансы быть выбранными

Обучение DAC

- Обучение на смешанных данных с разными частотами дискретизации приводит к тому, что модель «режет» высокие частоты
- Будем всегда добавлять в батч **полнополосные** (*full band*) примеры
- Вместо *Quantization Dropout* используется глобальный *Dropout* с $p = 0.5$
- Такой метод 50% времени учит модель работать идеально, а 50% — быть устойчивой
- **Функции потерь:**

Multi – Scale Mel loss

Adversarial Loss

Feature Mapping Loss

Hi-Fi Codec, 2023

Проблемы прошлых моделей:

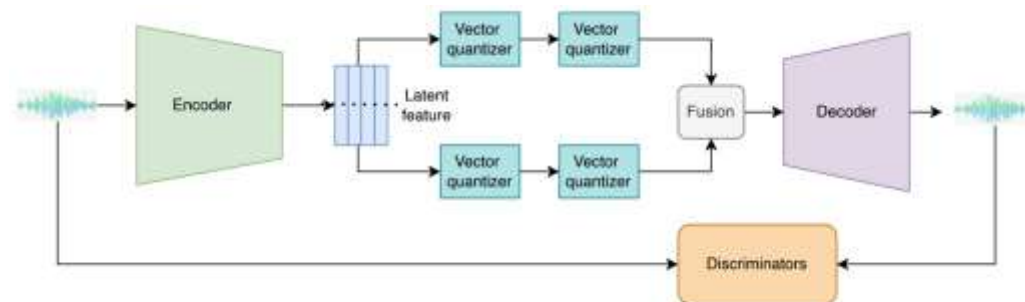
- В *RVQ* первый *codebook* несёт основную информацию, а последующие – лишь мелкие детали
- Информация в латентном пространстве размазана и разрежена
- Требуется много кодовых книг для хорошего качества, что создает длинные последовательности и перегружает модель

Идея Hi – Fi Codec:

- Вместо одной длинной цепочки *RVQ* распараллелим процесс квантования (***Group Residual Vector Quantization, GRVQ***)

Архитектура *Hi-Fi Codec*

- Латентный признак $\mathbf{z}$ разбивается на группы
- К каждой группе применяется свой набор *RVQ*
- На первых слоях *RVQ* сохраняется больше информации, что позволяет сократить их количество до 4
- **Encoder** представляет собой сверточную сеть + *LSTM*
- **Decoder** имеет аналогичную архитектуру с транспонированными свёртками



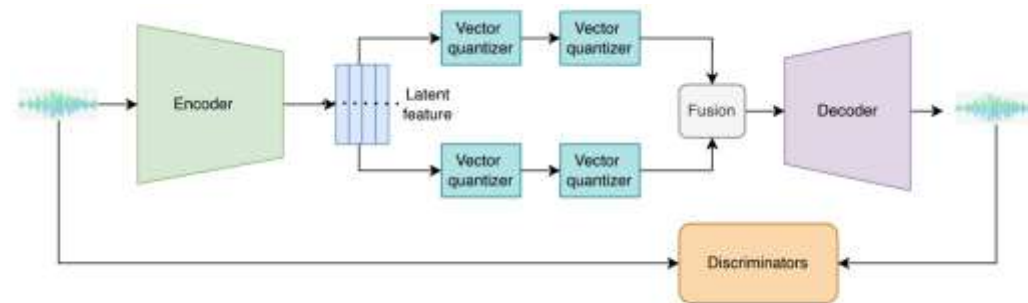
Обучение *Hi-Fi Codec*

Используется 3 вида дискриминаторов:

- ***MS-STFT*** для спектрограмм
- ***MPD*** для анализа периодичности аудио
- ***MSD*** для анализа аудио на разных масштабах

Функция потерь:

$$L = \lambda_{adv} \cdot L_{adv} + \lambda_{fear} L_{feat} + \lambda_{rec} L_{rec} + \lambda_c L_c$$



Результаты *Hi-Fi Codec*

- С 4 кодовыми книгами такое же качество реконструкции как у *Encodec* с 12
- По метрикам превосходит или равен *Encodec*
- Меньшее количество токенов облегчает обучение модели

Method	Sample rate (K hz)	Down-sample times	Number of codebooks	PESQ $\uparrow$	STOI $\uparrow$
Encodec (Facebook)	24	320	8	3.01	0.94
Encodec (Facebook)	24	320	12	3.21	0.95
Encodec (ours)	24	240	8	3.62	0.94
Encodec (ours)	24	32	2	3.08	0.91
Encodec (ours)	16	320	8	3.04	0.93
SoundStream (ours)	16	320	12	3.26	0.95
HiFi-Codec	24	240	4	3.63	0.95
HiFi-Codec	24	240	8	3.92	0.95
HiFi-Codec	24	320	4	3.64	0.95
HiFi-Codec	16	320	4	3.22	0.94

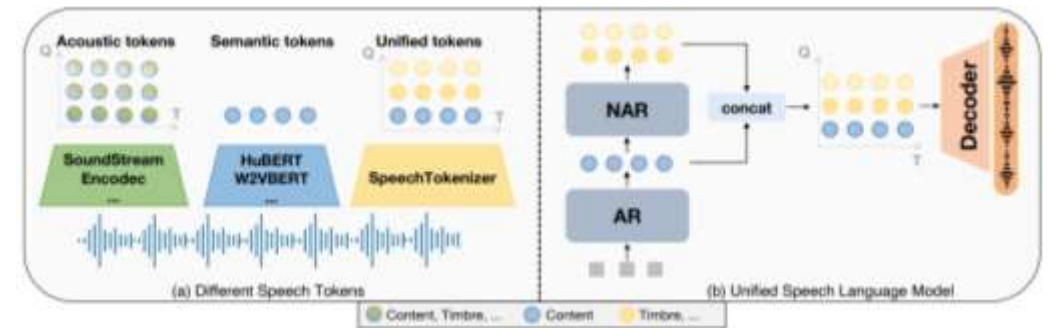
SpeechTokenizer, 2023

Проблемы прошлых моделей:

- Используют либо семантические (*HuBERT*), либо акустические токены (*Encodec*)
- Семантические токены хорошо выровнены с текстом, но теряют акустические детали (тембр, эмоции)
- Акустические токены хорошо передают качество звука, но плохо связаны с текстом

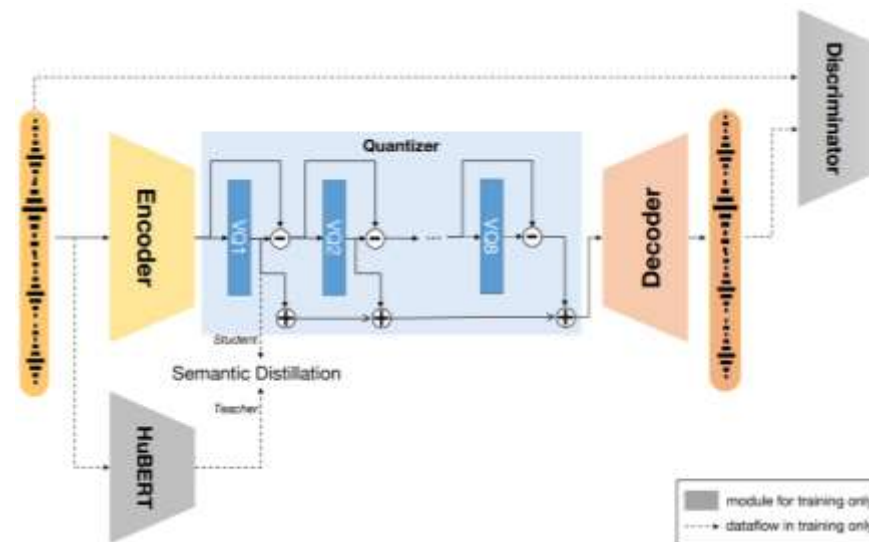
Идея *SpeechTokenizer*:

- Обучим модель, в которой акустическая и семантическая информации будут разделены



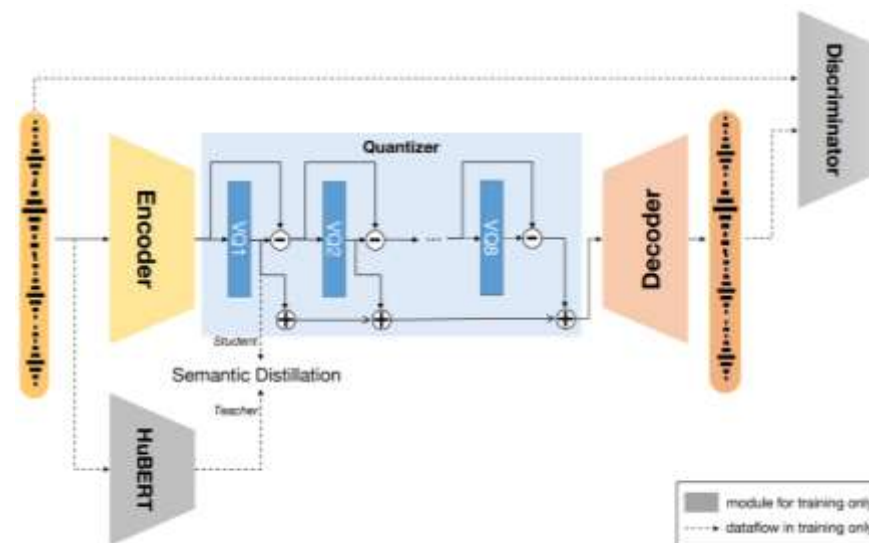
Архитектура *SpeechTokenizer*

- Модель представляет собой *encoder-decoder* архитектуру с использованием *RVQ*
- Информация внутри слоёв *RVQ* иерархически разделена (***Disentanglement***)
- Первый слой (*RVQ 1*) отвечает только за семантическую информацию
- Слои *RVQ 2* – *RVQ 8* отвечают за паралингвистическую информацию (тембр, просодия, шум)



Обучение *SpeechTokenizer*

- Вместо обычной *LSTM* в *Encoder* используется *BiLSTM*
- Для обучения используется *семантическая дистилляция*
- *RVQ 1* учится быть похожим на векторы *HuBERT* и сохранять семантическую информацию
- **Функция потерь:**
 - Reconstruction Loss (Wav, Mel)*
 - Adversarial Loss*
 - Commitment Loss*
 - Distillation Loss*



Результаты *SpeechTokenizer*

- Модель училась на датасетах *LibriSpeech*
- *WER* ниже, чем у *EnCodec*
- *MUSHRA* выше, чем у *EnCodec*
- Эксперименты показали, что токены первого слоя действительно семантические

Tokenizer	Objective		Subjective
	WER↓	VISQOL↑	MUSHRA↑
Groundtruth	4.58	-	91.46
EnCodec	5.11	4.37	79.86
SpeechTokenizer	5.04	4.30	90.55

WavTokenizer, 2024

Проблемы прошлых моделей:

- Для обучения генеративных моделей на основе *RVQ* нужно реализовывать сложные схемы, чтобы предсказывать сразу несколько токенов
- Для высокого качества звука *LLM* вынуждена генерировать сотни токенов в секунду
- Акустические токены плохо передают смысл

Идея WavTokenizer:

- Будем использовать *Single Vector Quantization* и добьёмся 40-75 токенов в секунду

Архитектура *WavTokenizer*

- *Encoder* состоит из свёрточных блоков + *LSTM*
- Размер кодовой книги увеличился до 4096
- Неиспользуемые коды заменяются на векторы из текущего батча
- *Decoder* имеет больше параметров, чем *Encoder*
- Вместо стандартного *upsampling* с помощью транспонированных свёрток используется *iSTFT* блоки
- В *Decoder* добавлен блок *Attention* и *ConvNeXt* блоки

Обучение *WavTokenizer*

Используется ансамбль дискриминаторов:

- *Multi-Period Discriminator* для анализа периодичности
- *Multi-Resolution Discriminator* для анализа спектра с разным разрешением
- *Multi-Scale STFT Discriminator* для анализа *STFT*

Функции потерь:

Reconstruction Loss (Mel)

Adversarial Loss

Feature Matching Loss

Quantizer Loss

Результаты *WavTokenizer*

- Модель училась на датасетах *LibriTTS*, *VCTK*, *Common Voice* и других (8000 часов)
- Превосходит *SOTA* модели по качеству
- Требуется в 10 раз меньше битрейта для достижения аналогичного или лучшего качества

Dataset	Model	Bandwidth	Nq	tokens	UTMOS	PESQ	STOI	VUV F1
LibriTTS test-clean	GT	-	-	-	4.0562	-	-	-
	DAC	9.0kpbs	9	900	3.9097	3.9082	0.9699	0.9781
	Encoder	6.0kpbs	8	600	3.0399	2.7202	0.9391	0.9527
	Vocos	6.0kpbs	8	600	3.6954	2.8069	0.9426	0.9437
	SpeechTokenizer	6.0kpbs	8	600	3.8794	2.6121	0.9165	0.9495
	DAC	4.0kpbs	4	400	3.4329	2.7378	0.9280	0.9572
	HiFi-Coder	3.0kpbs	4	400	3.7529	2.9611	0.9405	0.9617
	HiFi-Coder	4.0kpbs	4	300	3.9035	3.0116	0.9446	0.9576
	Encoder	3.0kpbs	4	300	2.3070	2.0517	0.9007	0.9198
	Vocos	3.0kpbs	4	300	3.5390	2.4026	0.9231	0.9358
	SpeechTokenizer	3.0kpbs	4	300	3.5632	1.9311	0.8778	0.9273
	DAC	1.0kpbs	1	100	1.8940	1.3464	0.7706	0.7941
	WavTokenizer	0.3kpbs	1	40	3.6016	1.7027	0.8615	0.9173
	WavTokenizer	0.9kpbs	1	75	4.0486	2.3730	0.9139	0.9382
LibriTTS test-other	GT	-	-	-	3.4831	-	-	-
	DAC	9.0kpbs	9	900	3.3566	3.7595	0.9576	0.9696
	Encoder	6.0kpbs	8	600	2.6568	2.6818	0.9241	0.9338
	Vocos	6.0kpbs	8	600	3.1956	2.5590	0.9209	0.9202
	SpeechTokenizer	6.0kpbs	8	600	3.2851	2.3269	0.8811	0.9205
	DAC	4.0kpbs	4	400	2.9448	2.5948	0.9083	0.9404
	HiFi-Coder	4.0kpbs	4	400	3.0750	2.5536	0.9126	0.9387
	HiFi-Coder	3.0kpbs	4	300	3.3034	2.6083	0.9166	0.9318
	Encoder	3.0kpbs	4	300	2.0883	2.0520	0.8835	0.8926
	Vocos	3.0kpbs	4	300	3.0558	2.1933	0.8967	0.9051
	SpeechTokenizer	3.0kpbs	4	300	3.0183	1.7373	0.8371	0.8907
	DAC	1.0kpbs	1	100	1.4986	1.3454	0.7505	0.7775
	WavTokenizer	0.3kpbs	1	40	3.0545	1.6622	0.8336	0.8953
	WavTokenizer	0.9kpbs	1	75	3.4312	2.2614	0.8907	0.9172
LJSpeech	GT	-	-	-	4.3794	-	-	-
	DAC	9.0kpbs	9	900	4.3007	3.9122	0.9733	0.9757
	Encoder	6.0kpbs	8	600	3.2286	2.6633	0.9441	0.9555
	Vocos	6.0kpbs	8	600	4.0332	2.9258	0.9497	0.9459
	SpeechTokenizer	6.0kpbs	8	600	4.2373	2.6413	0.9316	0.9452
	DAC	4.0kpbs	4	400	3.8109	2.7616	0.9338	0.9524
	HiFi-Coder	4.0kpbs	4	400	4.1656	2.7629	0.9446	0.9497
	HiFi-Coder	3.0kpbs	4	300	4.2692	2.9091	0.9485	0.9469
	Encoder	3.0kpbs	4	300	2.3905	2.0194	0.9058	0.9326
	Vocos	3.0kpbs	4	300	3.7880	2.5006	0.9310	0.9388
	SpeechTokenizer	3.0kpbs	4	300	3.9908	2.0458	0.9021	0.9299
	DAC	1.0kpbs	1	100	1.4438	1.2084	0.7822	0.8095
	WavTokenizer	0.3kpbs	1	40	4.0186	2.1142	0.9093	0.9406
	WavTokenizer	0.9kpbs	1	75	4.2580	2.4923	0.9312	0.9397

X-Codec, 2024

Проблемы прошлых моделей:

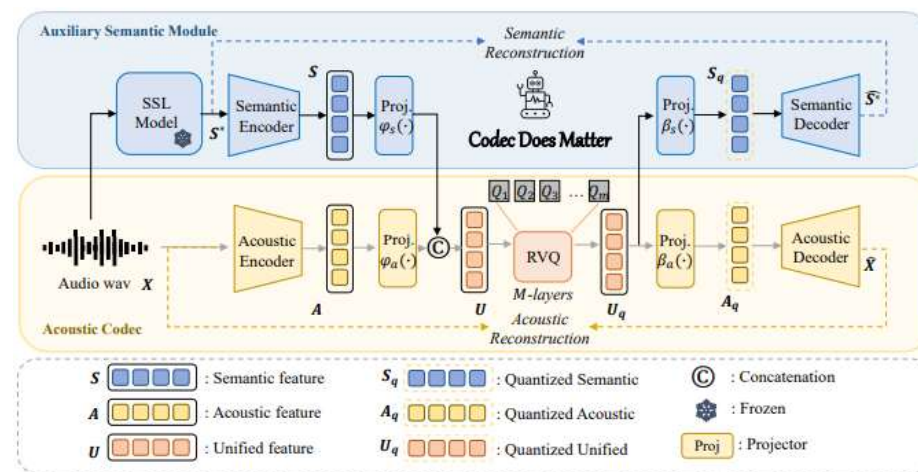
- Традиционные кодеки создавались для сжатия, но для *LLM* нужна и семантика
- *LLM* приходится предсказывать аудио без понимания контекста
- В результате возникают пропуски, искажение смысла, высокий *WER*
- Для *Speech Tokenizer* нужен этап дистилляции

Идея X – Codec:

- Объединим смысл и звук в один токен

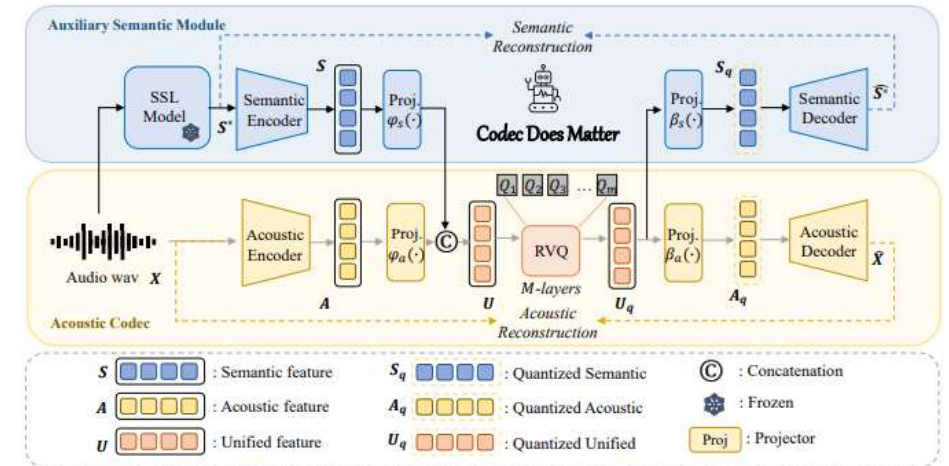
Архитектура X-Codec

- Модель имеет два входа слева и два выхода справа
- *Acoustic Encoder* сжимает аудио X в акустические признаки A
- *Auxiliary Semantic Module* использует предобученную *SSL* модель с признаками S^* , с помощью которой *Semantic Encoder* получает семантические признаки S
- Полученные признаки A и S проходят через линейные проекции *Proj* и объединяются в единый вектор U



Архитектура X-Codecs

- Объединенный вектор U проходит через RVQ
- Из полученных токенов U_q ***Acoustic Decoder*** восстанавливает исходное аудио
- Из этих же токенов ***Semantic Decoder*** восстанавливает семантические векторы S^*



Обучение X-Codec

- Для обучения используются 2 потока восстановления, чтобы гарантировать, что токены сохранили оба типа информации
- Используются функции потерь для семантики и для акустики:

Semantic Reconstruction Loss (MSE)

Mel Loss

STFT Loss

Adversarial Loss

Commitment Loss

- Для обучения используются 2 потока восстановления, чтобы гарантировать, что токены сохранили оба типа информации
- Используются функции потерь для семантики и для акустики:

Semantic Reconstruction Loss (MSE)

Mel Loss

STFT Loss

Adversarial Loss

Результаты X-Codec

- Модель училась на датасете **LibriTTS** в качестве *LLM* модели использовалась *VALL-E*
- По сравнению с *Encodec* и *DAC* значительно снизился *WER*
- Высокая похожесть на оригинал (*voice cloning*)
- Все автометрики лучше предыдущих кодеков

Codec	Training Data of Audio Codec	VALL-E AR stage			VALL-E AR+NAR stages		
		WER ↓	SIM-O ↑	UTMOS ↑	WER ↓	SIM-O ↑	UTMOS ↑
GT	-	2.23	0.67	4.10	2.23	0.67	4.10
Encodec [10]	C+DNS+AS+FSD+J	47.17	0.09	1.24	6.37	0.33	3.02
DAC [9]	C+DNS+V+AS+J+M	85.55	0.03	1.24	6.81	0.34	3.31
Speechtokenizer [13]	LibriSpeech	7.53	0.10	1.26	5.24	0.36	3.84
Baseline Acoustic Codec	LibriSpeech	22.32	0.16	3.15	7.70	0.41	3.89
X-Codec-hubert	LibriSpeech	5.27	0.22	3.85	4.07	0.42	4.16
X-Codec-wavlm-base-plus	MLS English	4.83	0.24	4.02	3.26	0.41	4.22

Спасибо за внимание!